Python Practical Programs & VIVA Qs

1. Write a Python program to work with the various ways of accessing the string. i) By using Indexing (Both Positive and Negative) ii) By using Slice Operator

```
s = "Python"
print("First char:", s[0])
print("Last char:", s[-1])
print("First 3 chars:", s[:3])
```

2. Write Python programs to print the following Patterns:

1

22

333

4444

55555

```
for i in range(1,6):
    print(str(i)*i)
```

A

A B

A B C

A B C D

A B C D E

```
list = ['A','B','C','D','E']
for i in range(len(list)):
    for j in range(i+1):
        print(list[j], end='')
    print()
```

*****

****

***

**

*

```
for i in range(6,0,-1):
    print("*" * i)
```

3. Write a Python program to work with the lambda functions in Python with suitable examples.

```
add = lambda a, b: a + b
print("Sum of 3 and 4:", add(3, 4))
```

4. Write a python program to generate the Fibonacci series.

```
def fibonacci(n):
    a, b = 0, 1
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
fibonacci(10)
```

5. Write a function that reverses the user defined value using python.

```
def reverse_string(s):
    return s[::-1]
print("Reversed:", reverse_string("hello"))
```

6. Write a function to check if the input value is Armstrong or not.

```
def is_armstrong(num):
    digits = str(num)
    power = len(digits)
    total = sum(int(d)**power for d in digits)
    return total == num
number = int(input("Enter a number: "))
if is_armstrong(number):
    print(number, "is an Armstrong number")
else:
    print(number, "is NOT an Armstrong number")
```

7. Write the function for the Input number is Palindrome or not.

```
def is_palindrome(s):
    return s == s[::-1]
print(is_palindrome("madam"))
```

8. Write the function for the Input number is Prime number or not.

```
def is_prime(num):
    if num <= 1:
        return False
    for i in range(2, num):
        if num % i == 0:
            return False
    return True
number = int(input("Enter a number: "))
```

```
if is_prime(number):
    print(number, "is a Prime number")
else:
    print(number, "is NOT a Prime number")
```

9. Write a recursive function to print the factorial for a given number.

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n-1)
print("Factorial of 5:", factorial(5))
```

10. Write a function that takes a character (i.e. a string of length 1) and returns True if it is a vowel, False otherwise.

```
def is_vowel(char):
    return char.lower() in 'aeiou'
print(is_vowel('a')) # True
print(is_vowel('b')) # False
```

11. Define a function that computes the length of a given list or string.

```
def compute_length(item):
    return len(item)
print("Length of list:", compute_length([1, 2, 3, 4]))
print("Length of string:", compute_length("hello"))
```

12. Write a Python program to print a specified list after removing the 0th, 2nd, 3rd and 5th elements.

```
def remove_indices(lst):
    indices = [0, 2, 3, 5]
    new_list = []

    for i in range(len(lst)):
        if i not in indices:
            new_list.append(lst[i])

    return new_list

print(remove_indices([10, 20, 30, 40, 50, 60, 70]))
```

13. Write a Python script to sort (ascending and descending) a dictionary by value

```
my_dict = {'a': 30, 'b': 10, 'c': 50, 'd': 20}

# Convert dictionary to list of pairs
items = list(my_dict.items())
```

```
# Sort ascending
items_sorted_asc = sorted(items, key=lambda x: x[1])

# Sort descending
items_sorted_desc = sorted(items, key=lambda x: x[1], reverse=True)

print("Ascending:", dict(items_sorted_asc))
print("Descending:", dict(items_sorted_desc))
```

14. Write a Python program to sum all the items in a dictionary

```
d = {'a': 10, 'b': 20, 'c': 30}
print("Sum of values:", sum(d.values()))
```

15. Implement the following Searching and Sorting Techniques question Python by using functions. i) Linear Search ii) Binary Search iii) Selection Sort iv) Bubble Sort v) Insertion vi) Merge Sort viii) Quick Sort

1. Linear Search

```
def linear_search(lst, key):
    for i in range(len(lst)):
        if lst[i] == key:
            return i
    return -1
```

2. Binary Search

```
def binary_search(lst, key):
    low = 0
    high = len(lst) - 1

    while low <= high:
        mid = (low + high) // 2

        if lst[mid] == key:
            return mid
        elif key < lst[mid]:
            high = mid - 1
        else:
            low = mid + 1

    return -1
```

3. Selection Sort

```
def selection_sort(lst):
    for i in range(len(lst)):
```

```
        min_index = i
        for j in range(i+1, len(lst)):
            if lst[j] < lst[min_index]:
                min_index = j
        lst[i], lst[min_index] = lst[min_index], lst[i]
    return lst
```

4. Bubble Sort

```
def bubble_sort(lst):
    n = len(lst)
    for i in range(n):
        for j in range(0, n - i - 1):
            if lst[j] > lst[j + 1]:
                lst[j], lst[j + 1] = lst[j + 1], lst[j]
    return lst
```

5. Insertion Sort

```
def insertion_sort(lst):
    for i in range(1, len(lst)):
        key = lst[i]
        j = i - 1

        while j >= 0 and key < lst[j]:
            lst[j + 1] = lst[j]
            j -= 1

        lst[j + 1] = key

    return lst
```

6. Merge Sort

```
def merge_sort(lst):
    if len(lst) <= 1:
        return lst

    mid = len(lst) // 2
    left = merge_sort(lst[:mid])
    right = merge_sort(lst[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
```

```
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

7. Quick Sort

```
def quick_sort(lst):
    if len(lst) <= 1:
        return lst
    pivot = lst[0]
    left = [x for x in lst[1:] if x < pivot]
    right = [x for x in lst[1:] if x >= pivot]
    return quick_sort(left) + [pivot] + quick_sort(right)
```

Sample list to test all functions

```
numbers = [50, 20, 40, 10, 30]
```

16. Generate different plotting using PyLab. Such as Line plot, Bar chart, Pie chart, Histogram, Scatter plot.

Install Command

```
pip install matplotlib
```

PyLab Program (All Plots)

```
from pylab import *
# Sample data
x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 25, 30]

# Line Plot
figure()
plot(x, y)
title("Line Plot")
xlabel("X-axis")
ylabel("Y-axis")
show()

# Bar Chart
figure()
bar(x, y)
title("Bar Chart")
```

```
show()

# Pie Chart
figure()
labels = ['A', 'B', 'C', 'D', 'E']
pie(y, labels=labels, autopct='%1.1f%%')
title("Pie Chart")
show()

# Histogram
figure()
data = [10, 20, 10, 30, 40, 20, 30, 10]
hist(data)
title("Histogram")
show()

# Scatter Plot
figure()
scatter(x, y)
title("Scatter Plot")
xlabel("X-axis")
ylabel("Y-axis")
show()
```

17. Write a Python script to plotting a curve

Install Command

```
pip install matplotlib
```

Plot a Curve

```
import matplotlib.pyplot as plt
# X and Y values for curve
x = [1, 2, 3, 4, 5]
y = [1, 4, 9, 16, 25]    # y = x² curve

plt.plot(x, y)
plt.title("Simple Curve Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

18. Write a Python program to implement 0/1 knapsack algorithm

```
def knapsack(values, weights, W):
    n = len(values)
    # Create DP table
    dp = [[0] * (W + 1) for _ in range(n + 1)]
```

```
    # Fill the table
    for i in range(1, n + 1):
        for w in range(W + 1):
            # If item can fit
            if weights[i - 1] <= w:
                dp[i][w] = max(
                    dp[i - 1][w],
                    dp[i - 1][w - weights[i - 1]] + values[i - 1]
                )
            else:
                dp[i][w] = dp[i - 1][w]
    return dp[n][W]
# Example
vals = [60, 100, 120]
wts = [10, 20, 30]
W = 50
print(knapsack(vals, wts, W))
```

19. Write a Python program to use divide and conquer algorithm

```
def merge_sort(arr):
    # Base condition
    if len(arr) <= 1:
        return arr
    # Divide
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    # Conquer (merge)
    return merge(left, right)
def merge(left, right):
    result = []
    i = j = 0
    # Merge two sorted halves
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    # Add remaining elements
    result.extend(left[i:])
    result.extend(right[j:])
    return result
# Example
numbers = [38, 27, 43, 3, 9, 82, 10]
print("Sorted list:", merge_sort(numbers))
```

20. Write a Python program of GUI to ask confirmation or give alerts using dialogs.

```
import tkinter as tk
from tkinter import messagebox as mb
root = tk.Tk()
def on_quit():
    if mb.askyesno('Verify','Really quit?'):
        mb.showinfo('Quit','Exiting now')
    root.destroy()
tk.Button(root, text='Quit', command=on_quit).pack()
root.mainloop()
```

21. Write a Python program of GUI to create a simple calculator

```
import tkinter as tk
def calculate():
    try:
        res = eval(entry.get())
        result_var.set(str(res))
    except:
        result_var.set("Err")
root = tk.Tk()
root.title("Simple Calculator")
entry = tk.Entry(root)
entry.pack()
tk.Button(root, text='=', command=calculate).pack()
result_var = tk.StringVar()
tk.Label(root, textvariable=result_var).pack()
root.mainloop()
```

22. Write a Python program to get a list of existing databases.

Install Command

```
pip install mysql-connector-python
```

Code:

```
import mysql.connector
cnx =
mysql.connector.connect(user='root',password='pwd',host='127.0.0.1')
cursor = cnx.cursor()
cursor.execute('SHOW DATABASES')
for db in cursor:
    print(db)
```

23. Write a Python program to create, insert, update, delete, and select data from tables of database. With exceptional handling.

Install Command

```
pip install mysql-connector-python
```

Code:

```
import mysql.connector as m

# Function to connect MySQL
def get_con():
    return m.connect(
        host="localhost",
        user="root",
        password="",      # default for XAMPP MySQL
        database="python"
    )

# ---------------------------- CREATE TABLE ----------------------------
-
def create_table():
    con = get_con()
    cr = con.cursor()
    try:
        q = """
        CREATE TABLE IF NOT EXISTS std (
            id INT AUTO_INCREMENT PRIMARY KEY,
            Name VARCHAR(50),
            City VARCHAR(50),
            Stream VARCHAR(50)
        )
        """
        cr.execute(q)
        con.commit()
        print("Table 'std' created or already exists.\n")
    except Exception as e:
        print("Error:", e)
    finally:
        con.close()

# ---------------------------- INSERT ----------------------------
def insert_record():
    con = get_con()
    cr = con.cursor()

    name = input("Enter Name: ")
    city = input("Enter City: ")
    stream = input("Enter Stream: ")

    try:
```

```
        q = "INSERT INTO std (Name, City, Stream) VALUES
('{}','{}','{}')".format(
            name, city, stream
        )
        cr.execute(q)
        con.commit()
        print("Record Inserted\n")
    except Exception as e:
        con.rollback()
        print("Error:", e)
    finally:
        con.close()

# ---------------------------- UPDATE ----------------------------
def update_record():
    con = get_con()
    cr = con.cursor()

    id = int(input("Enter ID to update: "))

    try:
        cr.execute("SELECT * FROM std WHERE id = {}".format(id))
        row = cr.fetchone()

        if row:
            print("Record Found:", row)

            name = input("Enter New Name: ")
            city = input("Enter New City: ")
            stream = input("Enter New Stream: ")

            q = "UPDATE std SET Name='{}', City='{}', Stream='{}' WHERE
id={}".format(
                name, city, stream, id
            )
            cr.execute(q)
            con.commit()
            print("Record Updated\n")
        else:
            print("ID not found\n")

    except Exception as e:
        con.rollback()
        print("Error:", e)
    finally:
        con.close()

# ---------------------------- DELETE ----------------------------
```

```python
def delete_record():
    con = get_con()
    cr = con.cursor()

    id = int(input("Enter ID to delete: "))

    try:
        q = "DELETE FROM std WHERE id = {}".format(id)
        cr.execute(q)

        if cr.rowcount > 0:
            con.commit()
            print("Record Deleted\n")
        else:
            print("ID not found\n")

    except Exception as e:
        con.rollback()
        print("Error:", e)
    finally:
        con.close()

# ---------------------------- SELECT ----------------------------
def select_records():
    con = get_con()
    cr = con.cursor()

    try:
        cr.execute("SELECT * FROM std")
        rows = cr.fetchall()

        print("\n----- All Records -----")
        for r in rows:
            print(r)
        print("-----------------------\n")

    except Exception as e:
        print("Error:", e)
    finally:
        con.close()

# ---------------------------- MAIN MENU ----------------------------
while True:
    print("========== MENU ==========")
    print("1. Create Table")
    print("2. Insert Record")
    print("3. Update Record")
    print("4. Delete Record")
```

```
    print("5. View All Records")
    print("6. Exit")
    print("=========================")
    choice = input("Enter choice: ")
    if choice == "1":
        create_table()
    elif choice == "2":
        insert_record()
    elif choice == "3":
        update_record()
    elif choice == "4":
        delete_record()
    elif choice == "5":
        select_records()
    elif choice == "6":
        print("Exiting...")
        break
    else:
        print("Invalid choice\n")
```

-------Viva Question & Answer-----

Unit 1

1. What is Python?
Python is a high-level, interpreted, general-purpose programming language.

2. Why is Python called a high-level language?
Because it hides low-level machine details and is easy to read.

3. What are variables in Python?
Variables are names used to store data values.

4. What is type casting?
Converting one data type into another.

5. What are Python keywords?
Reserved words in Python with predefined meanings.

6. What is indentation in Python?
Whitespace used to define blocks of code.

7. What is a conditional statement?
A statement that executes code based on conditions.

8. Difference between if, elif, and else.
if checks first condition; elif checks next; else runs if none match.

9. What is nested if?
An if statement inside another if.

10. What are logical operators?
Operators like and, or, not.

11. What is a string in Python?
A sequence of characters enclosed in quotes.

12. How do you take user input?
Using the input() function.

13. Difference between input() and raw_input().
raw_input() was in Python 2; input() is used in Python 3.

14. What is string slicing?
Accessing parts of a string using indices.

15. Name 5 string functions.
upper(), lower(), strip(), split(), replace().

16. What is a loop?
A structure that repeats code multiple times.

17. Difference between for and while loop?
for iterates over sequences; while runs until condition becomes false.

18. What is a nested loop?
A loop inside another loop.

19. What is break?
Stops the loop entirely.

20. What is continue?
Skips the current iteration.

21. What is a function?
A reusable block of code.

22. Built-in vs user-defined functions?
Built-in are provided by Python; user-defined are created by users.

23. What is scope?
Region of code where a variable is accessible.

24. What are local and global variables?
Local inside function; global outside any function.

25. What is recursion?
Function calling itself.

26. What is a module?
A Python file containing functions and variables.

27. How to import a module?
Using import module_name.

28. What is a file?
A storage resource used to save data.

29. Difference between text and binary files?
Text stores characters; binary stores raw bytes.

30. File modes in Python?
r, w, a, rb, wb, ab.

31. What is a list?
A mutable collection of elements.

32. What is a tuple?
An immutable collection.

33. Difference between list and tuple?
Lists can change; tuples cannot.

34. What is mutability?
Ability to change contents.

35. Are strings mutable?
No, strings are immutable.

36. What are first-class functions?
Functions treated as values.

37. What is a lambda function?
A small anonymous function.

38. What is a dictionary?
A key-value pair collection.

39. How to add/remove items in dictionary?
Use dict[key]=value; del dict[key].

40. List vs dictionary?
Lists use index; dictionaries use keys.

Unit 2

41. What is an exception?
An error detected during execution.

42. What is exception handling?
Managing runtime errors using try-except.

43. Explain try, except, finally.
try tests code; except handles errors; finally runs always.

44. What is raise?
Manually throwing an exception.

45. Exceptions as control flow?
Using exceptions to alter program flow.

46. What is an assertion?
A sanity-check using assert keyword.

47. When to use assert?
During debugging to verify conditions.

48. What is a class?
A blueprint for objects.

49. What is an object?
An instance of a class.

50. What is a constructor?
Method that initializes object state.

51. What is __init__()?
Default constructor in Python.

52. What is self?
Represents the current object.

53. What is inheritance?
Acquiring properties of another class.

54. Types of inheritance?
Single, multiple, multilevel, hierarchical.

55. What is method overriding?
Redefining parent class method in child.

56. What is multiple inheritance?
A class inheriting from multiple parents.

57. What is encapsulation?
Hiding data inside a class.

58. Public, private, protected?
public: *, protected: _var, private: __var.

59. What is linear search?
Searching sequentially.

60. What is binary search?
Search by dividing sorted list in halves.

61. Selection sort vs bubble sort?
Selection picks minimum; bubble swaps adjacent.

62. What is a hashtable?
Structure storing data using hashing.

Unit 3

63. What is PyLab?
Module in matplotlib used for plotting.

64. Which library is used for graphs?
matplotlib.

65. Draw line graph?
Using plot(x,y).

66. What is a figure?
Main container of plots.

67. What is dynamic programming?
Solving problems by storing previous results.

68. Fibonacci sequence?
Series where each number is sum of previous two.

69. 0/1 Knapsack problem?
Optimization choosing items with weight limit.

70. Divide and conquer?
Breaking problems into smaller parts.

Unit 4

71. What is a protocol?
Rules for communication.

72. What is a socket?
Endpoint for network communication.

73. What is an IP address?
Unique address of device on network.

74. What is URL?
Address of resource on internet.

75. Read webpage source?
Using urllib or requests.

76. Download image?
Using requests and saving file.

77. TCP vs UDP?
TCP is connection-oriented; UDP is connectionless.

78. What is TCP server?
Server that communicates using TCP.

79. What is TCP client?
Client communicating via TCP.

80. UDP server & client?
Server/client using UDP protocol.

81. Two-way communication?
Bidirectional data exchange.

82. Send email in Python?
Using smtplib.

83. What is Tkinter?
Python's built-in GUI library.

84. Event-driven programming?
Code runs on events.

85. What is a widget?
GUI component.

86. Examples of widgets?
Label, Button, Entry.

87. pack(), grid(), place()?
Layout managers.

88. Create simple window?
Using Tk() constructor.

89. What is TreeView?
Widget to display tabular data.

90. What are frames?
Containers for grouping widgets.

Unit 5

91. Install MySQL connector?
Using pip install mysql-connector-python.

92. What is a cursor?
Object to execute SQL queries.

93. Connect Python to MySQL?
Using mysql.connector.connect().

94. Fetch all rows?
Using cursor.fetchall().

95. Insert data?
Using INSERT query.

96. Update record?
Using UPDATE query.

97. Delete record?
Using DELETE query.

98. What is parameterized query?
Query using placeholders to prevent SQL injection.

99. Create table using Python?
Using CREATE TABLE SQL statement.

100. What is commit()?
Saves changes to database.